JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번 202118025 이름: 김종호 제출일: 2026-10-28

서비스 이름 술슐랭 (SULSHULLANG) — 술집 추천·리뷰 게시판

기술 조합과 버전 Vue 3.5.43 (Vite 8.3.4) / Express 5.3.0 (jsonwebtoken 9.0.3, sql.js 1.14.2) / Windows 11, Node.js v24.22.1, Chrome 155

GitHub 저장소 URL https://github.com/KJH-ssaljibadeulraemi/sulshullang-jwt-board

소스 코드 기준 main 브랜치 / 구현 기준 커밋 70c06b1 (최종 제출 커밋 SHA 는 LMS 에 제출)

술슐랭은 직접 가 본 술집을 별점과 짧은 후기로 공유하는 게시판입니다. 친구들과 2 차 장소를 정할 때 매번 검색만 하다 보니, 실제로 가 본 사람의 후기를 모아 보면 좋겠다는 생각에서 주제를 정했습니다. 대상은 술집을 고르는 대학생이나 직장인이고, 로그인하지 않아도 후기를 볼 수 있지만 글쓰기는 로그인한 사람만, 수정·삭제는 글쓴이 본인만 할 수 있습니다. 별점 입력칸을 따로 만들면 API 명세(title·content 만 허용)에 맞지 않아서, 본문 첫 줄에 '★ 4.5 / 5'처럼 적는 방식으로 정했습니다.

기술은 수업 예제와 같은 Vue + Express 를 골랐습니다. 수업 코드와 구조가 같아서 이해하기 쉬웠고, 프론트와 백엔드를 둘 다 JavaScript 로 다룰 수 있어서 한 언어에 집중할 수 있었습니다.

제공 템플릿에서 바꾼 부분은 인증 TODO 였던 backend-express/src/auth.js 의 함수들(BE-01~03), frontend-vue/src/api.js(FE-01~04), App.vue 의 스크립트, 그리고 화면 문구와 서버 시작 로그입니다. 라우트, 입력 검사, DB 저장, 비밀번호 검증 같은 나머지 제공 코드는 그대로 두었습니다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	app.js GET /api/posts, /api/posts/:id (제공 코드)	비로그인 200 확인 (T01)
로그인과 JWT 발급	완료	auth.js loadSecret, loadTtl, issueToken	HS256, 기본 TTL 90 초
인증과 내 정보 조회	완료	auth.js currentUserId / app.js GET /api/me	토큰 없음·변조·만료를 각각 다른 401 code 로 구분
본인 글 작성·수정·삭제	완료	auth.js assertOwner / app.js POST·PUT·DELETE	남의 글 403, 작성자 필드 위조 400
만료·로그아웃·오류 표시	완료	frontend-vue/src/api.js, App.vue	새로고침하면 로그인 풀림(요구사항대로), 로그아웃해도 이미 복사된 토큰은 만료 전까지 유효
DB 데이터 유지	완료	store.js (제공), backend-express/data/board.sqlite	서버 재시작 후 유지 확인 (T15)

구조와 변경 내역

전체 연결 구조

```
[브라우저: Vue 화면 127.0.0.1:4173]
    |  /api/... 요청 (fetch)
    ▼
[Vite preview 프록시]  vite.config.js 의 preview.proxy
    |  http://127.0.0.1:8000 으로 전달
    ▼
[Express API 서버]  app.js 라우트 → auth.js 인증·권한 → store.js
    |
    ▼
[SQLite 파일]  backend-express/data/board.sqlite
```

로그인 요청 (토큰 없이 보냄): 로그인 버튼 → api.js 의 login() → POST /api/auth/login 에 email·password 전송 → 서버에서 fields()로 입력 형식 확인 → verifyCredentials()로 비밀번호 확인 → issueToken()이 JWT 발급 → 200 응답으로 accessToken, expiresIn, user 가 돌아옴 → api.js 가 이 값을 session 변수(브라우저 메모리)에 저장합니다. localStorage 나 sessionStorage 에는 저장하지 않습니다.

게시글 수정 요청 (토큰을 같이 보냄): 수정 저장 버튼 → api.update() → PUT /api/posts/{id}, 이때 헤더에 Authorization: Bearer <토큰>이 붙습니다 → 서버는 currentUserId()로 토큰 확인(실패 401) → postId()로 ID 형식 확인(400) → getPost()로 글이 있는지 확인(404) → assertOwner()로 글쓴이인지 확인(403) → fields()로 제목·내용 확인(400) → store.update()로 DB 파일에 저장 → 200. 화면에서는 401 이 오면 로그아웃 처리하고, 403 이 오면 로그인은 그대로 두고 안내만 띄웁니다.

로그인한 상태에서는 api.js 가 모든 /api 요청에 토큰을 붙이고, 서버는 /api/me 와 글 등록·수정·삭제에서만 토큰을 검사합니다. 로그인하지 않았으면 Authorization 헤더 자체를 만들지 않고, 목록·상세는 토큰 없이도 볼 수 있습니다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
backend-express/src/auth.js loadSecret, loadTtl (새로 추가)	없음 (TODO 주석만 있었음)	.env 의 JWT_SECRET(64 자리 16 진수)을 32 바이트 키로 바꾸고, 비어 있거나 형식이 틀리거나 예시 값 그대로면 서버가 켜지지 않게 함. TOKEN_TTL 숫자 확인(기본 90)	비밀 키를 환경설정으로 넣고 검사하라는 요구사항 (AI 초안을 그대로 적용, 테스트로 확인)
auth.js / issueToken	501 NOT_IMPLEMENTED 를 던지는 임시 코드	HS256 으로 sub·iss·aud·exp 가 들어간 JWT 발급	BE-01 (AI 초안을 그대로 적용, 테스트로 확인)
auth.js / currentUserId	항상 데모 ID 1 을 돌려줌	Bearer 토큰 꺼내기 → jwt.verify 로 검증 → exp·sub 확인 → 실제 계정 확인 → ID 반환. 실패하면 401 세 가지로 구분	BE-02 (AI 초안을 그대로 적용, 테스트로 확인)
auth.js / assertOwner	아무것도 검사하지 않는 빈 함수	로그인한 사람 ID 와 글의 authorId 가 다르면 403 FORBIDDEN	BE-03 (AI 초안을 그대로 적용, 테스트로 확인)
frontend-vue/src/api.js authorizationHeaders, request, login, canEdit	빈 헤더, 로그인 미구현, canEdit 는 항상 true	토큰을 메모리에 저장, 토큰 있을 때만 Bearer 헤더, 오류 code 별 안내 문구, 401 이면 로그인 정보 삭제, 204 는 JSON 파싱 안 함, 작성자 비교	FE-01~04 (AI 초안을 그대로 적용, 테스트로 확인)

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
frontend-vue/src/App.vue <script>의 run, save, logout	성공 문구 고정, 비로그인 처리 없음	404 면 상세를 비우고 목록 새로고침, 비로그인이면 글쓰기 대신 안내, 로그아웃 시 입력칸 비움	FE-03·FE-04 (AI 초안을 그대로 적용, 테스트로 확인)
App.vue 화면 부분, index.html	'우리의 기록' 제목, 인증 미완성 데모 배너	술술랭 제목·부제·배너로 변경, 비로그인 안내 한 줄 추가, 탭 제목 변경	서비스 주제 반영, 데모 배너 교체 (문구는 직접 작성)
backend-express/src/app.js 시작 로그	'— 인증 TODO 미완성' 출력	'— 술술랭 JWT 인증 서버'로 변경	구현을 끝냈으므로 미완성 문구 제거 (직접 수정)

출처를 정리하면, app.js 의 라우트와 입력 검사, store.js, users.js, errors.js, vite.config.js 는 제공 코드를 그대로 썼습니다. 화면 문구, 시작 로그 문구, .env 설정은 제가 직접 했습니다. auth.js 의 함수들, api.js, App.vue 스크립트는 AI(Claude)가 준 초안을 그대로 붙여 넣고 테스트로 동작을 확인했습니다(5 절 참고). 쓰지 않는 템플릿(backend-fastapi, backend-spring, frontend-react)은 제출 저장소에서 지웠고, 실제 .env 와 DB 파일은 .gitignore 로 올라가지 않게 했습니다.

로컬 실행 기록

- **환경:** Windows 11, Node.js v24.22.1, Chrome 155
- **백엔드:** backend-express 폴더에서 npm ci → Copy-Item .env.example .env → node -e "console.log(require('node:crypto').randomBytes(32).toString('hex'))"로 만든 키를 JWT_SECRET 에 넣음 → npm start (http://127.0.0.1:8000)
- **프론트:** frontend-vue 폴더에서 npm ci → npm run build (✓ built in 249ms) → npm run preview (http://127.0.0.1:4173)
- **SQLite 파일:** backend-express/data/board.sqlite (서버를 켜면 자동으로 만들어짐)
- **4173 접속·API 연결:** 빌드한 화면에서 목록 200, 로그인 200, 등록 201, 수정 200, 삭제 204 가 정상으로 나오는 것을 확인했습니다 (BUILD_1~5)
- **재시작 후 유지:** 백엔드를 끈 상태에서 새로고침하면 502 가 나왔고, 다시 npm start 후 새로고침하니 글·작성자·작성 시각이 그대로 있었습니다 (T15_1~4)
- **새 폴더 clone 재현:** C:\clone-test 에 GitHub 저장소를 새로 clone 한 뒤 README 순서대로 백엔드(npm ci → .env 키 설정 → npm start)와 프론트(npm ci → npm run build → npm run preview)를 실행했습니다. 처음엔 글이 0 개인 빈 DB 가 자동으로 만들어졌고, Alice 로 로그인해서 글 등록까지 정상으로 됐습니다. (2026-10-28, CLONE_reproduce_4173.png)
- **증거 위치:** docs/evidence/ (자세한 실행 방법은 저장소 README.md)

핵심 코드 설명

3 1 로그인과 JWT 발급

파일 / 함수: backend-express/src/auth.js — issueToken / backend-express/src/app.js — POST /api/auth/login (제공 코드, 발췌하면서 줄바꿈만 정리)

코드 발췌

```
// backend-express/src/auth.js
export function issueToken(user) {
  return jwt.sign({}, SECRET, {
    algorithm: "HS256",
    subject: String(user.id), // sub: 문자열 사용자 ID
    issuer: ISSUER,          // iss
    audience: AUDIENCE,      // aud
    expiresIn: TTL,          // exp = 지금 + TTL 초
  });
}
// backend-express/src/app.js (제공 코드)
  const input = fields(req.body, { email: 254, password: 128 });
  const user = verifyCredentials(input.email, input.password);
  if (!user)
    throw new ApiError(401, "INVALID_CREDENTIALS", "이메일 또는 비밀번호를 확인하세요.");
  const accessToken = issueToken(user);
  res.json({ accessToken, expiresIn: Number(process.env.TOKEN_TTL || 90), user });
```

동작과 응답 코드 설명

입력: 로그인 요청 본문의 email 과 password 입니다. fields()가 이 두 필드만 왔는지, 문자열인지, 비어 있지 않은지, 길이가 맞는지 확인하고 틀리면 **400 BAD_INPUT** 을 보냅니다.

처리: 제공된 verifyCredentials()가 비밀번호를 비교하고, 맞는 계정일 때만 issueToken()이 호출됩니다. 토큰에는 sub(사용자 ID 문자열), iss, aud, exp 만 들어가고 비밀번호나 해시는 넣지 않았습니다. 서명에 쓰는 SECRET 은 loadSecret()이 서버 시작 때 .env 의 64 자리 16 진수를 32 바이트로 바꿔 만든 값이고, 키가 비었거나 형식이 틀리면 서버가 켜지지 않습니다. 실제로 키를 잘못 넣었을 때 서버가 시작을 거부했습니다(BE01_secret_reject_on_bad_key).

출력: 성공하면 **200** 과 {accessToken, expiresIn: 90, user: {id, name, email}}이 옵니다. 비밀번호가 틀리거나 없는 이메일이면 둘 다 **401 INVALID_CREDENTIALS** 인데, 응답 크기도 92 Bytes 로 똑같아서 어떤 이메일이 가입돼 있는지 알 수 없습니다. 발급받은 토큰의 가운데 부분을 풀어 보니 sub 가 "1", iss 가 webservice-class, aud 가 board-api 였고 exp − iat 가 90 이라 설정이 맞게 들어간 것을 확인했습니다.

3 2 브라우저 토큰 저장과 전송

파일 / 함수: frontend-vue/src/api.js — session, authorizationHeaders, request, login

코드 발췌

```
let session = { accessToken: "", user: null };
function authorizationHeaders() {
  return session.accessToken
    ? { Authorization: `Bearer ${session.accessToken}` }
    : {};
}
// request() 안의 응답 처리 부분
  let data = null;
  if (response.status !== 204) {
    data = await response.json().catch(() => null);
  }
  if (!response.ok) {
    const code = data?.code;
    if (response.status === 401 && code !== "INVALID_CREDENTIALS") {
      clearSession();
    }
    // ... ERROR_MESSAGES[code]로 안내 문구를 만들어 throw (중략)
export async function login(email, password) {
  const data = await request("/auth/login", {
    method: "POST",
    body: JSON.stringify({ email, password }),
  });
  session = { accessToken: data.accessToken, user: data.user };
  return data.user;
}
```

동작과 응답 코드 설명

저장: 로그인에 성공하면 accessToken 과 user 를 api.js 의 session 변수에만 넣습니다. 그래서 새로고침하면 사라지는데, 과제 조건이 브라우저 메모리에 보관하는 것이라 정상입니다(T16). 화면은 이 user 로 'Alice 님 로그인 중'을 보여 줍니다.

전송: authorizationHeaders()는 토큰이 있을 때만 Authorization: Bearer <토큰>을 만들고, 없으면 빈 객체를 돌려줘서 'Bearer undefined' 같은 헤더가 생기지 않습니다. 개발자 도구 Network 에서 /api/me 요청에 Bearer 헤더가 붙은 것(FE02)과 로그아웃 뒤에는 Authorization 헤더가 없는 것(T11)을 확인했습니다.

응답 코드 처리: 200·201 은 받은 결과로 목록과 상세를 다시 그립니다. **204** 는 본문이 없어서 JSON 을 읽지 않습니다. **400** 은 안내만 띄우고 입력칸은 지우지 않습니다. **401** 중 TOKEN_REQUIRED·INVALID_TOKEN·TOKEN_EXPIRED 는 clearSession()으로 토큰을 지워 화면이 로그아웃 상태가 되고 다시 로그인하라는 안내가 나옵니다. 로그인 실패(**401 INVALID_CREDENTIALS**)는 원래 로그인 전이니까 지울 필요가 없어서 제외했습니다. **403** 은 토큰은 멀쩡하고 권한만 없는 것이라 로그인을 유지합니다. **404** 는 상세를 비우고 목록을 다시 불러옵니다.

로그아웃: App.vue 의 logout()이 clearSession()을 부르고 화면의 사용자 정보와 입력칸을 바로 비웁니다. 서버에 이미 나간 토큰을 취소하는 기능은 아닙니다.

서버 인증과 작성자 권한

3 3 서버 JWT 검증

파일 / 함수: backend-express/src/auth.js — currentUserId (주석 일부 생략)

코드 발췌

```
export function currentUserId(req) {
  const header = req.get("authorization") ?? "";
  const match = /^Bearer (\S+)$/.exec(header);
  if (!match) {
    throw new ApiError(401, "TOKEN_REQUIRED", "로그인이 필요합니다.");
  }
  let payload;
  try {
    payload = jwt.verify(match[1], SECRET, {
      algorithms: ["HS256"], issuer: ISSUER, audience: AUDIENCE,
    });
  } catch (error) {
    if (error.name === "TokenExpiredError") {
      throw new ApiError(401, "TOKEN_EXPIRED", "로그인이 만료되었습니다. 다시 로그인하세요.");
    }
    throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.");
  }
  if (typeof payload.exp !== "number") { /* 401 INVALID_TOKEN */ }
  if (typeof payload.sub !== "string" || !/^[1-9]\d*$/.test(payload.sub)) { /* 401 INVALID_TOKEN */ }
  const userId = Number(payload.sub);
  if (!findPublic(userId)) {
    throw new ApiError(401, "INVALID_TOKEN", "존재하지 않는 사용자입니다.");
  }
  return userId;
}
```

동작과 응답 코드 설명

입력: 요청 헤더의 Authorization 값입니다. ‘Bearer 토큰’ 형식이 아니거나 헤더가 아예 없으면 **401 TOKEN_REQUIRED** 입니다.

처리: 토큰 내용을 그냥 읽는 decode 가 아니라 jwt.verify 로 검사합니다. algorithms 를 HS256 하나로 정해서 다른 방식의 토큰은 받지 않고, 서버 키로 서명이 맞는지, iss · aud 가 맞는지, 만료됐는지를 라이브러리가 확인합니다. exp 가 아예 없는 토큰은 verify 가 만료 검사를 하지 않아서 따로 막았고, sub 가 양의 정수 문자열인지, 그 ID 의 계정이 실제로 있는지도 확인합니다. 이걸 다 통과한 ID 만 /api/me 와 글 등록 · 수정 · 삭제로 넘어갑니다.

출력: 통과하면 사용자 ID 를 돌려주고, /api/me 는 **200** 과 {id, name, email}만 응답합니다(비밀번호 · 해시 없음). 실패하면 정상 서명인데 시간이 지난 토큰은 **401 TOKEN_EXPIRED**, 서명이 틀리거나 클레임이 이상하거나 없는 사용자면 **401 INVALID_TOKEN** 입니다. 401 응답에는 app.js 의 공통 오류 처리 부분이 WWW-Authenticate: Bearer 헤더를 붙여 줍니다.

확인: 토큰 없음(T04), 서명 첫 글자를 바꾼 토큰(T09), 90 초 지난 토큰(T10)을 보내서 각각 TOKEN_REQUIRED, INVALID_TOKEN, TOKEN_EXPIRED 로 다르게 나오는 것을 확인했습니다. 알고리즘이나 iss/aud 가 다른 토큰, exp · sub 가 빠진 토큰은 직접 만들어 보지는 못했습니다.

3 4 작성자 권한 검사

파일 / 함수: backend-express/src/auth.js — assertOwner / app.js — PUT /api/posts/:id (제공 코드, 번호 주석은 설명을 위해 보고서에만 추가)

코드 발췌

```
// backend-express/src/auth.js
export function assertOwner(userId, authorId) {
  if (Number(userId) !== Number(authorId)) {
    throw new ApiError(403, "FORBIDDEN", "본인이 작성한 글만 수정·삭제할 수 있습니다.");
  }
}
// backend-express/src/app.js (제공 코드)
app.put("/api/posts/:id", (req, res) => {
  const userId = currentUserId(req),  // ① 토큰 확인 401
    id = postId(req),                 //    ID 형식 400
    p = getPost(id);                  // ② 글 존재 404
  assertOwner(userId, p.authorId);    // ③ 글쓴이 확인 403
  res.json(store.update(id, fields(req.body, { title: 100, content: 5000 })));  // ④ 입력 400 → ⑤ 저장
});
```

동작과 응답 코드 설명

비교하는 값: currentUserId()가 토큰에서 확인한 '지금 요청한 사람'의 ID 와, DB 에 저장돼 있는 글의 authorId(처음 글을 쓴 사람)를 비교합니다. 다르면 **403 FORBIDDEN** 을 던지는데, store.update() · store.remove()보다 먼저 실행되기 때문에 DB 는 바뀌지 않습니다. DELETE 라우트도 같은 순서입니다.

401 과 403 의 차이: 401 은 '누군지 확인이 안 됨'(토큰 없음 · 변조 · 만료)이고, 403 은 '누군지는 알지만 이 글에 대한 권한이 없음'입니다. Bob 의 정상 토큰으로 Alice 글을 고치면 403 이지만, Bob 토큰이 만료됐으면 ①에서 먼저 401 이 됩니다. 또 Bob 이 Alice 글에 공백 제목을 보냈을 때도 ③이 ④보다 먼저라서 400 이 아니라 403 이 나왔습니다.

작성자 위조 방지: 글을 등록할 때는 store.create(…, userId)로 토큰에서 확인한 ID 를 작성자로 저장하고, 요청 본문에 authorId 나 userId 가 들어 있으면 fields()가 **400 BAD_INPUT** 으로 막습니다(T12). 화면에서 남의 글 버튼을 숨기는 canEdit 는 보기 편하게 하는 것일 뿐이고, 실제로 막는 건 서버의 이 검사입니다.

확인: Bob 토큰으로 Alice 글을 PUT · DELETE 했을 때 둘 다 403 이었고, 그 뒤 다시 조회해 보니 제목 · 내용 · updatedAt 이 그대로였습니다. 본인 글은 수정 200(작성자 · 작성일 유지), 삭제 204 가 나왔습니다.

테스트 결과

실행 환경: Windows 11 / Node.js v24.22.1 / Chrome 155 / 백엔드 npm start(8000), 프론트 npm run dev(5173)와 npm run build + npm run preview(4173) / API 요청은 VS Code 확장 Thunder Client, 화면은 Chrome 개발자 도구 Network 로 확인

로컬 배포 확인: frontend-vue 에서 npm run build 성공(✓ built in 249ms) → npm run preview → http://127.0.0.1:4173 에서 로그인·등록·수정·삭제와 작성자별 버튼 확인(BUILD_1~5). 새 폴더 clone 재현은 아직 안 함(제출 전에 할 예정)

테스트 일시 및 코드 버전: 2026-10-28 00:30 ~ 19:25 / 저장소 main 브랜치 커밋 70c06b1 과 같은 코드

토큰 TTL: 기본 90 초 (빠진 캡처를 찍을 때만 잠깐 600 초) / 만료 화면 테스트 15 초 / 자동 만료 검사 3 초 → 제출 설정은 90 초

실제 결과 칸은 Thunder Client 와 Network 에서 직접 본 상태 코드와 code 입니다. 직접 캡처하지 못한 세부 항목은 제공된 자동 검사(tools/check_api.py) 결과로 확인했고 '(자동)'이라고 적었습니다. 증거 파일은 docs/evidence/에 있습니다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	비로그인, 4173 에서 목록·상세 조회	200, 공개 조회	200 / 해당 없음 (목록·상세 보임)	PASS	T01_public_list_4173 FE04_logout_buttons_hidden
T02	Alice 로그인 (Thunder Client, 화면)	200, JWT·user, 로그인 표시	200 / 해당 없음, expiresIn 90, 'Alice 님 로그인 중'	PASS	T02_login_alice_200 FE01_alice_logged_in
T03	틀린 비밀번호 / 없는 이메일로 로그인	401 INVALID_CREDENTIALS, 토큰 미발급	401 / INVALID_CREDENTIALS (두 경우 모두, 토큰 없음)	PASS	T03_login_wrong_password_401 T03_login_unknown_email_401
T04	토큰 없이 GET /me, POST /posts	각각 401 TOKEN_REQUIRED	/me 401 / TOKEN_REQUIRED, www-authenticate: Bearer POST 401 / TOKEN_REQUIRED (자동)	PASS	T04_me_no_token_401 T04_..._www_authenticate_header
T05	Alice 토큰으로 글 작성 후 /me	201, authorId 1, /me 는 Alice	201 / 해당 없음, authorId 1, location /api/posts/12 /me 200 Alice	PASS	T05_alice_create_post_201 T05_create_post_201_location_header T05_me_alice_200

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T06	Alice 가 본인 글 수정·삭제	200, 204, DB 반영	수정 200 (제목 '수정 테스트'로 바뀜) 삭제 204, 본문 0 Bytes	PASS	BUILD_4_alice_update_after_relogin_4173 T06_alice_delete_own_post_204
T07	Bob 정상 토큰으로 Alice 글 수정	403, 원문 유지	403 / FORBIDDEN, 다시 조회해도 원문·updatedAt 그대로 공백 제목으로 수정해도 403	PASS	T07_bob_put_alice_post_403 T07_T08_alice_post_unchanged_after_403 T07_bob_blank_title_on_alice_post_403
T08	Bob 정상 토큰으로 Alice 글 삭제	403, 글 유지	403 / FORBIDDEN, 글 남아 있음	PASS	T08_bob_delete_alice_post_403
T09	서명 첫 글자를 바꾼 토큰으로 /me·변경	401 INVALID_TOKEN, 변경 없음	/me 401 / INVALID_TOKEN POST·PUT·DELETE 401 / INVALID_TOKEN (자동)	PASS	T09_tampered_signature_me_401 AUTO_check_final_40_passed
T10	짧은 TTL 로 받은 토큰이 만료된 뒤 /me·변경	401 TOKEN_EXPIRED, 화면 로그아웃	/me 401 / TOKEN_EXPIRED (90 초 경과) TTL 15: 글 등록 401, 화면 로그아웃, 글 안 생김 TTL 3: 만료 4 건 (자동)	PASS	T10_expired_token_me_401_ttl90 T10_login_expiresIn_15 T10_expired_ui_post_401_logged_out
T11	로그인 → 로그아웃 → 내 정보 조회	헤더에 토큰 없음, 401 TOKEN_REQUIRED	401, Authorization 헤더 없음, '로그인이 필요합니다' 안내 (TOKEN_REQUIRED 문구)	PASS	T11_logout_then_me_401 T11_logout_request_no_authorization_header
T12	Bob 토큰 + authorId:1 / userId:1 로 등록	400 BAD_INPUT, 위조 글 미생성	둘 다 400 / BAD_INPUT	PASS	T12_reject_authorId_400 T12_reject_userId_400
T13	빈 제목·공백 본문·길이 초과 등록	400 BAD_INPUT, DB 변경 없음	공백 제목 400 / BAD_INPUT, 글 개수 그대로, 입력값 유지 공백 본문·101 자·5001 자 400 (자동)	PASS	T13_bad_input_400_network T13_bad_input_400_message

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T14	없는 ID 조회·수정·삭제, 잘못된 ID 조회	없는 글 404, 잘못된 ID 400	GET 99999 404 / NOT_FOUND, GET abc 400 / BAD_INPUT PUT 99999(정상 토큰) 404 / NOT_FOUND DELETE 99999 404 (자동)	PASS	T14_not_found_99999_404 T14_invalid_id_abc_400 T14_valid_token_put_missing_post_404
T15	글 작성 후 백엔드 종료·재실행	글·작성자·내용 유지	종료 중 조회 502, 재시작 후 200, 글 2 개와 작성자·작성 시각 그대로	PASS	T15_1 ~ T15_4
T16	로그인 후 새로고침	글 유지, 로그인 해제	200 / 해당 없음, 글 그대로, '로그아웃 상태'	PASS	T16_1_logged_in_before_refresh T16_2_after_refresh_...
T17	토큰 없이 수정·삭제, Bob 본인 작성·수정·삭제	토큰 없음 401, Bob 본인 작업 성공	PUT·DELETE 401 / TOKEN_REQUIRED Bob 등록 201, 수정 200, 삭제 204	PASS	T17_no_token_put_401, T17_no_token_delete_401 T17_bob_own_create/update/delete_4173

HTTP 응답 확인 기록

아래 값은 Thunder Client 의 Status 와 Response Headers, Chrome Network 의 Status Code 에서 직접 본 값입니다.

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	200 (목록·상세·로그인·/me·수정)	T01, T02, T05_me, BUILD_4
새 글 등록	**201 및 Location**	201 / location: /api/posts/12	T05_create_post_201_location_header
본인 글 삭제	**204 및 빈 본문**	204 / 본문 0 Bytes	T06_alice_delete_own_post_204, FE03
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	400 / BAD_INPUT (공백 제목, authorId, userId, ID abc)	T13, T12, T14_invalid_id_abc_400
잘못된 계정	**401 INVALID_CREDENTIALS**	401 / INVALID_CREDENTIALS	T03 (2 건)
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	401 / TOKEN_REQUIRED (토큰 없음). 헤더 형식 오류는 따로 안 해 봄	T04, T11, T14_no_token, T17_no_token
변조·클레임 오류	**401 INVALID_TOKEN**	401 / INVALID_TOKEN (서명 변조). 클레임 오류는 따로 안 해 봄	T09, AUTO_check_final
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	401 / TOKEN_EXPIRED	T10_expired_token_me_401_ttl90, AUTO_check_expiry
다른 작성자의 변경	**403 FORBIDDEN**	403 / FORBIDDEN (수정·삭제·공백 제목 수정)	T07, T08
없는 글	**404 NOT_FOUND**	404 / NOT_FOUND (조회, 정상 토큰으로 수정)	T14_not_found_99999_404, T14_valid_token_...

WWW-Authenticate: 토큰 없이 GET /api/me 를 보냈을 때 401 응답의 Response Headers 에 www-authenticate: Bearer 가 있었습니다(T04_me_no_token_www_authenticate_header). app.js 의 공통 오류 처리 부분이 401 일 때 이 헤더를 붙입니다.

401 과 403 일 때 화면: 401(만료·토큰 없음)이 오면 메모리의 토큰이 지워져서 로그인 칸이 '로그아웃 상태'로 바뀌고, 글쓰기 칸에 '글을 쓰려면 먼저 로그인하세요'가 나왔습니다(T10, T11). 403 은 로그인을 유지한 채 '본인이 작성한 글만 수정·삭제할 수 있습니다'만 띄우도록 만들었습니다. 다만 화면에서는 남의 글에 수정·삭제 버튼이 아예 안 보여서 403 화면을 직접 띄워 보지는 못했고, 403 은 Thunder Client 로 직접 요청해서 확인했습니다.

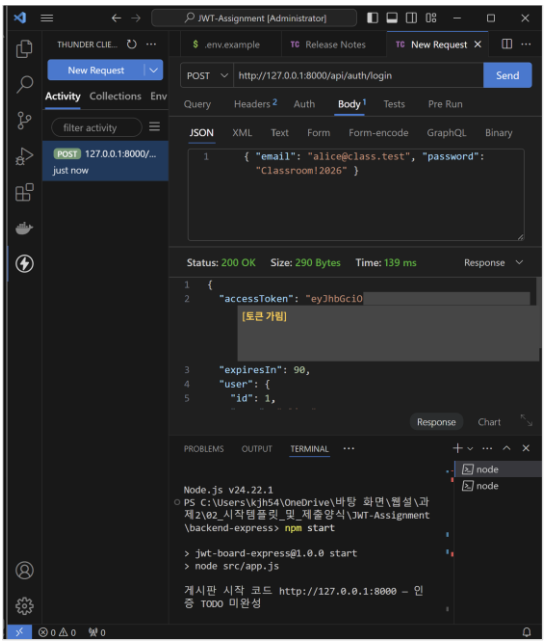
실패 후 데이터 그대로인지: Bob 의 PUT·DELETE 가 403 으로 막힌 뒤 토큰 없이 GET /api/posts/1 을 조회하니 200 으로 제목 '전주 한라 이자카야', 내용, updatedAt(작성 시각과 같음)이 그대로였습니다(T07_T08_alice_post_unchanged_after_403). 자동 검사의 'failed request leaves data unchanged'도 모두 PASS 였습니다.

조건이 겹칠 때: 토큰 없이 PUT /api/posts/99999 를 보내면 **401 / TOKEN_REQUIRED**, Alice 의 정상 토큰으로 같은 요청을 보내면 **404 / NOT_FOUND** 가 나왔습니다. 글이 있는지보다 토큰 검사를 먼저 한다는 것을 확인했습니다(T14_no_token_put_missing_post_401, T14_valid_token_put_missing_post_404).

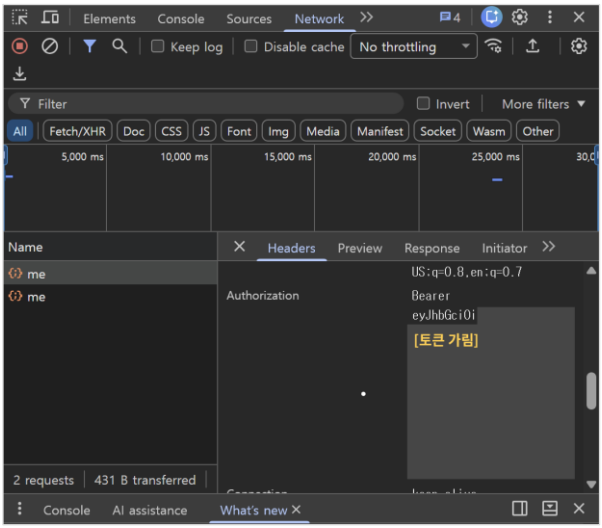
테스트 증거

로그인과 토큰 전송

증거 1 — 로그인 성공(토큰은 앞부분만 남기고 가림)과 /api/me 요청에 붙은 Authorization 헤더



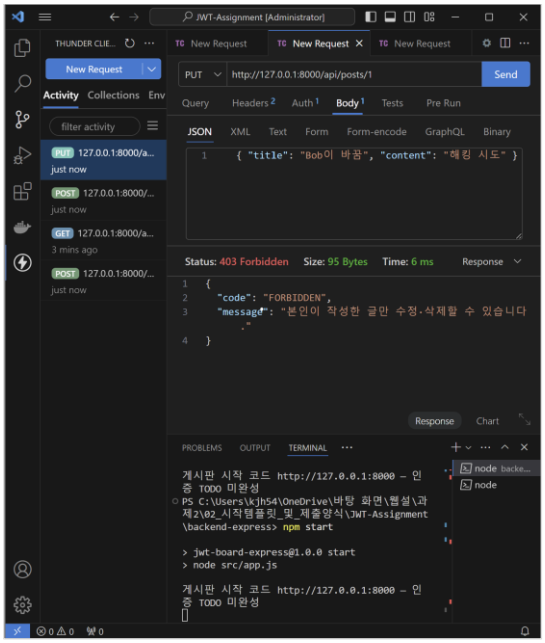
T02 로그인 200, expiresIn 90



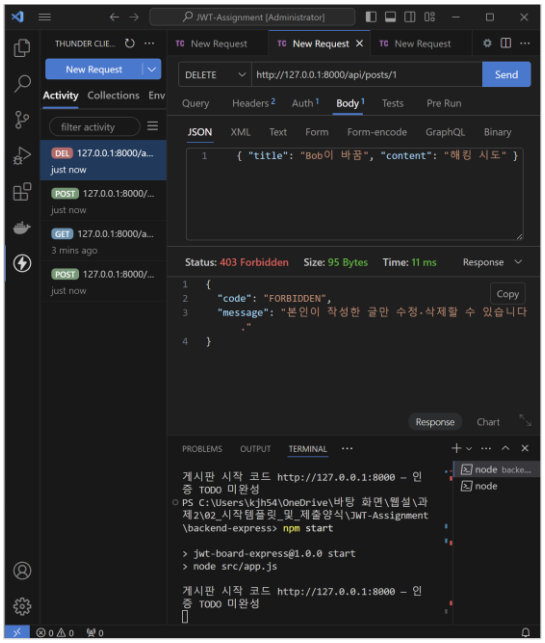
FE-02 요청 헤더 Authorization: Bearer

다른 작성자의 수정과 삭제

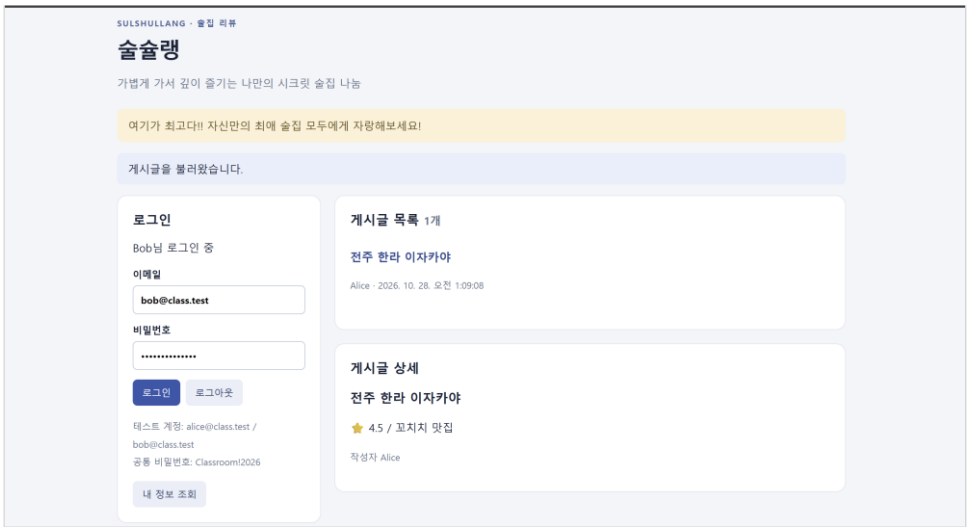
증거 2 — Bob 의 정상 토큰으로 Alice 글(id 1) 수정·삭제 시도



T07 PUT → 403 FORBIDDEN



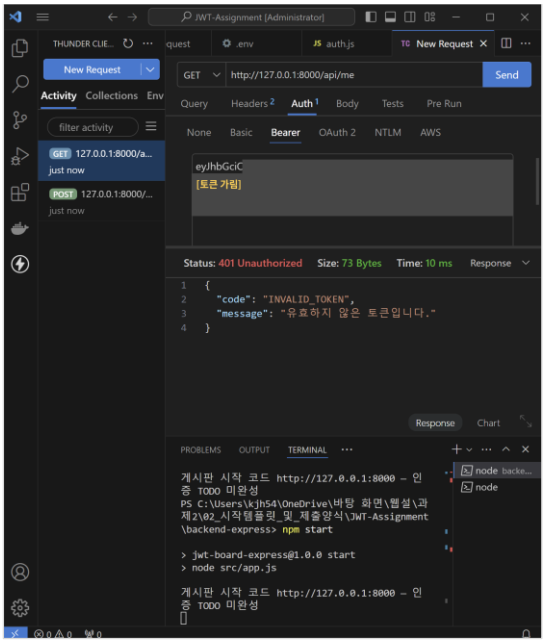
T08 DELETE → 403 FORBIDDEN



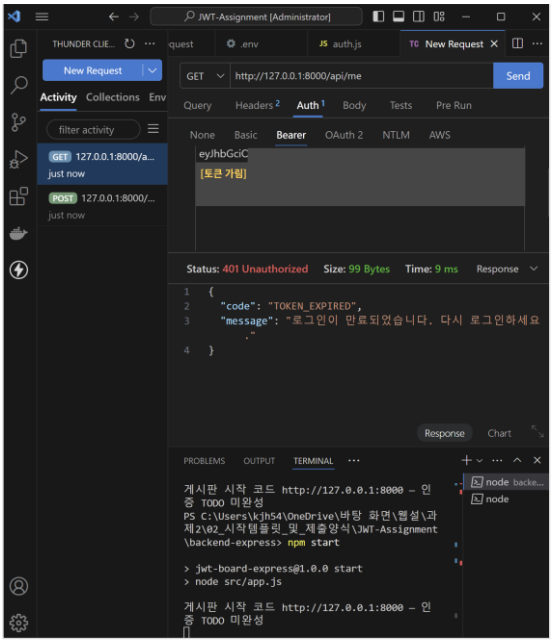
화면: Bob 으로 로그인하면 Alice 글에는 수정·삭제 버튼이 없음 (서버 403 과는 별개)

토큰 변조와 만료

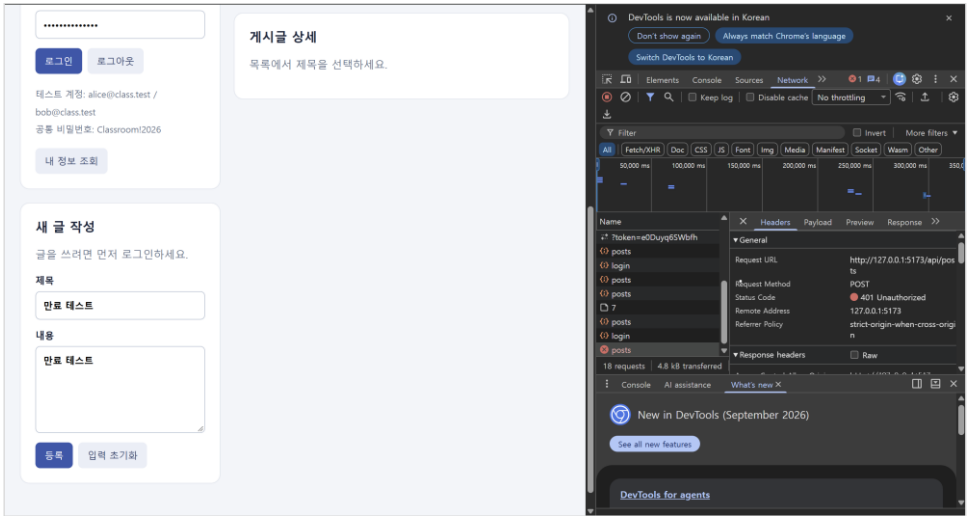
증거 3 — 서명을 바꾼 토큰과 만료된 토큰의 서버 응답, 만료 뒤 화면



T09 서명 변조 → 401 INVALID_TOKEN



T10 만료 → 401 TOKEN_EXPIRED



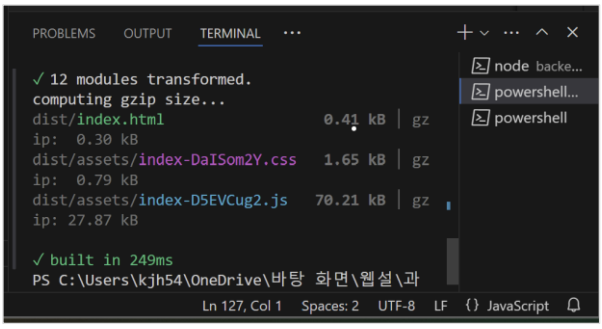
T10 TTL 15 초, 만료 후 등록 → 401, 화면이 로그아웃 상태로 바뀌고 입력값은 남아 있음

자동 검사와 로컬 배포 증거

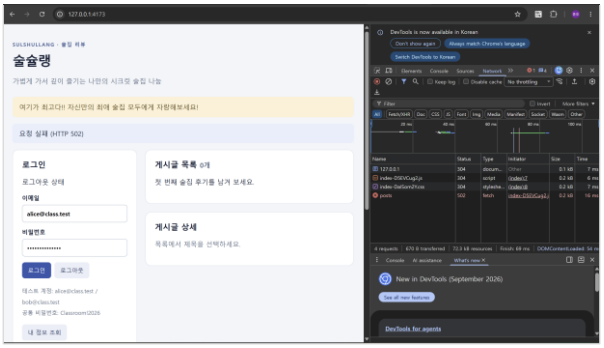
자동 검사: 백엔드를 켜고 저장소 루트에서 py tools/check_api.py --mode final 을 실행해 All 40 checks passed 가 나왔습니다. TOKEN_TTL=3 으로 서버를 다시 켠 뒤 --expiry 옵션을 붙여 실행했을 때는 All 48 checks passed 였습니다. 자동 검사는 API 응답만 확인하기 때문에 화면, 새로고침, 서버 재시작은 위처럼 따로 직접 확인했습니다. 전체 로그는 docs/evidence/check_final.txt, check_expiry.txt 에 있습니다.



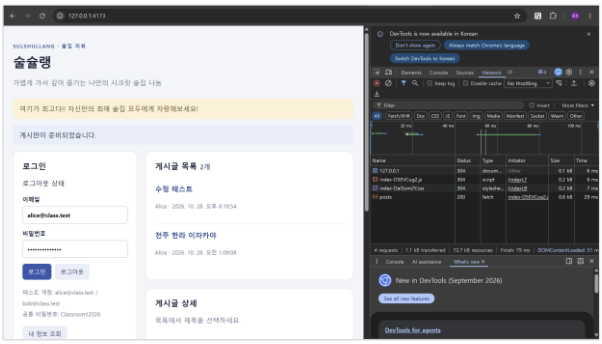
자동 검사 --expiry: 48 개 통과



npm run build 성공



T15 백엔드 종료 중 조회 502



T15 재시작 후 4173 에서 글 유지

AI 활용과 참고 자료

AI 사용 여부: 사용

과제 전체를 진행하면서 Claude(Anthropic)를 사용했습니다. Node.js · Git 설치와 VS Code 사용법부터 막혀서 순서대로 물어보며 진행했고, 인증 부분 코드(auth.js, api.js, App.vue 스크립트)는 Claude 가 준 초안을 거의 그대로 붙여 넣었습니다. 대신 붙여 넣은 뒤에는 Thunder Client 와 개발자 도구로 직접 요청을 보내 응답 코드와 화면을 하나씩 확인했고, 그 결과를 4 절에 적었습니다. 대화 전체는 제출하지 않고 대표 사례 3 건만 아래에 적었습니다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
Claude (claude.ai, 앱에 표시된 모델 Claude Opus 5.5)	생성 / 테스트	auth.js 의 loadSecret · loadTtl · issueToken · currentUserId · assertOwner (보고서 3.1 · 3.3 · 3.4)	그대로 반영, 테스트로 동작 확인
Claude	생성 / 테스트	frontend-vue/src/api.js, App.vue 스크립트와 비로그인 안내 한 줄 (보고서 3.2)	그대로 반영, 테스트로 동작 확인
Claude	학습 / 디버깅	개발 환경 설치, .env 키 설정, 서버가 안 켜지는 오류와 TTL 이 안 바뀌는 문제의 원인, 테스트 방법(Thunder Client · Network)	참고
Claude	문장정리	README.md, 이 보고서 초안, 증거 캡처 파일 이름 정리와 토큰 가리기	초안을 받아 읽어 보고 수정 후 반영

대표 사례

도구 / 날짜: Claude / 2026-10-28

요청한 내용: “BE-02 토큰 검증은 어떻게 구현해야 해?” 하고 단계별로 알려 달라고 했습니다.

AI 가 제안한 내용: Authorization 헤더에서 Bearer 뒤의 토큰을 꺼내고, jwt.verify 에 algorithms: ["HS256"]와 issuer · audience 를 넣어 검증하고, TokenExpiredError 면 TOKEN_EXPIRED, 나머지 오류는 INVALID_TOKEN 으로 나누고, exp · sub 형식과 실제 계정이 있는지 확인한 뒤 숫자 ID 를 돌려주는 코드를 받았습니다.

실제 반영 위치: backend-express/src/auth.js currentUserId (보고서 3.3)

내가 확인하고 수정한 내용: 받은 코드를 그대로 붙여 넣었고, 안내대로 파일 맨 위에 findPublic 을 가져오는 import 한 줄을 추가했습니다. 코드를 직접 고친 부분은 없고, 붙여 넣기 전에 각 줄이 무엇을 하는지 설명을 같이 받아서 읽어 봤습니다.

검증 방법과 결과: 토큰 없이 /me → 401 TOKEN_REQUIRED(T04), Alice 토큰으로 /me → 200(T05), 서명 첫 글자를 바꾼 토큰 → 401 INVALID_TOKEN(T09), 90 초가 지난 토큰 → 401 TOKEN_EXPIRED(T10)를 Thunder Client 로 직접 보내 확인했습니다. 자동 검사도 40 개 · 48 개 모두 통과해서 동작한다고 판단했습니다.

도구 / 날짜: Claude / 2026-10-28

요청한 내용: 만료 화면을 테스트하려고 TOKEN_TTL 을 15 로 바꾸고 20 초 기다렸다가 글을 썼는데, 401 이 안 나오고 글이 그냥 등록됐다고 물어봤습니다.

AI 가 제안한 내용: 로그인 응답의 expiresIn 값을 보면 서버가 실제로 쓰는 TTL 을 알 수 있고, 값이 바뀌지 않았다면 .env 를 저장만 하고 서버를 다시 켜지 않은 것이니 재시작한 뒤 다시 로그인해서 해 보라는 답을 받았습니다.

실제 반영 위치: 테스트 절차, backend-express/.env (보고서 4 절 T10)

내가 확인하고 수정한 내용: 서버는 켜질 때만 .env 를 읽는다는 걸 이때 처음 알았습니다. 잘못 등록된 시험 글은 지우고, 백엔드를 재시작했습니다. 코드는 고칠 필요가 없었습니다.

검증 방법과 결과: 재시작 후 다시 로그인해서 Network 의 login 응답에 expiresIn 이 15 로 나온 것을 확인했고, 20 초 뒤 글을 등록하니 POST /api/posts 가 401 로 거부되고 화면이 로그아웃 상태로 바뀌었으며 글은 생기지 않았습니다(T10_login_expiresIn_15, T10_expired_ui_post_401_logged_out).

도구 / 날짜: Claude / 2026-10-28

요청한 내용: BE-01 코드를 넣고 npm start 를 했더니 'JWT_SECRET 이 없거나 형식이 잘못되었습니다'라는 오류가 나면서 서버가 안 켜진다고 물어봤습니다.

AI 가 제안한 내용: 이 오류는 키가 잘못됐을 때 서버가 일부러 시작을 거부하는 것이라는 설명과 함께, 키 값을 화면에 출력하지 않고 길이와 형식만 확인하는 명령, 그리고 키를 다시 만들어 .env 에 공백이나 따옴표 없이 한 줄로 넣는 방법을 받았습니다. .env.example 에는 실제 키를 넣으면 안 된다는 것도 들었습니다.

실제 반영 위치: backend-express/.env, .env.example, auth.js loadSecret (보고서 3.1)

내가 확인하고 수정한 내용: 키를 다시 넣어서 해결했습니다. 그런데 이때 .env.example 에도 실제 키를 넣었다가, 공개 저장소에 키가 올라갈 수 있다는 지적을 받고 원래 예시 값(REPLACE_WITH_RANDOM_64_HEX_CHARACTERS)으로 되돌렸습니다.

검증 방법과 결과: 키가 잘못됐을 때는 서버가 켜지지 않았고(BE01_secret_reject_on_bad_key), 키를 고친 뒤에는 정상으로 켜져서 로그인이 200 으로 됐습니다(T02). GitHub 에 올리기 전 커밋 목록에 .env 가 없는 것도 확인했습니다.

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
수업 제공 시작 코드와 과제 문서 (docs/01~07, tools/check_api.py)	전체	라우트·DB 저장·비밀번호 검증은 제공 코드 그대로 사용, 요구사항과 테스트 시나리오를 기준으로 구현·확인
jsonwebtoken (제공된 의존성) https://github.com/auth0/node-jsonwebtoken	auth.js	jwt.sign / jwt.verify 와 옵션(algorithm, subject, issuer, audience, expiresIn) 사용
Claude (Anthropic) https://claude.ai	위 표 참고	AI 활용 내역은 이 절에 따로 기록, 받은 코드는 직접 테스트해서 확인

이해도 확인과 회고

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

요청 본문의 authorId 를 그대로 믿으면 Bob 이 authorId: 1 을 넣어서 Alice 가 쓴 것처럼 글을 만들 수 있습니다. 그러면 누가 글을 썼는지를 믿을 수 없게 되는데. 제 코드는 글을 등록할 때 본문 값이 아니라 currentUserId()가 토큰에서 확인한 ID 로 작성자를 저장하고, 본문에 authorId 나 userId 같은 필드가 오면 fields()가 400 BAD_INPUT 으로 거부합니다. Bob 토큰으로 authorId: 1, userId: 1 을 각각 보내서 400 이 나오는 것을 확인했습니다(T12).

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

버튼은 화면에서 안 보이게 하는 것뿐이라, Thunder Client 나 curl 같은 도구로 PUT · DELETE 요청을 직접 보내면 버튼이 없어도 서버까지 요청이 갑니다. 그래서 진짜 막는 건 서버에서 해야 합니다. 제 코드에서는 auth.js 의 assertOwner()가 토큰의 사용자 ID 와 DB 에 저장된 글의 authorId 를 비교해서 다르면 403 을 보냅니다. 실제로 화면에는 버튼이 없는 상태에서 Bob 토큰으로 직접 요청을 보내 403 을 확인했습니다(T07, T08).

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode 는 토큰 안의 내용을 읽기만 하고 서명이 맞는지는 확인하지 않습니다. 그래서 누가 sub 를 다른 사람 ID 로 바꾼 토큰을 보내도 그대로 믿게 됩니다. verify 는 서버 키로 서명을 확인하고 만료 시간 같은 것도 같이 검사합니다. 제 코드는 jwt.verify 에 algorithms: ["HS256"], issuer, audience 를 넣어서 알고리즘, 서명, 만료, iss, aud 를 검사하고, 그 뒤에 exp 가 있는지, sub 가 숫자 형식인지, 그 계정이 실제로 있는지까지 확인한 다음 ID 를 넘깁니다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 이 탭의 메모리에 있던 토큰과 사용자 정보를 지워서, 다음 요청부터 Authorization 헤더가 안 붙게 합니다(T11). 토큰 만료는 서버가 exp 시간이 지난 걸 보고 401 TOKEN_EXPIRED 로 거부하는 것이고, 화면은 이걸 받으면 로그인 상태를 풀어 줍니다(T10). 새로고침은 메모리 변수가 초기화돼서 로그인이 풀리지만, 글은 DB 파일에 있어서 그대로 남습니다(T16). 이미 어딘가에 복사된 토큰은 서버가 따로 기억하지 않기 때문에, 로그아웃을 해도 만료 시간(최대 90 초)까지는 계속 쓸 수 있습니다.

5. 가장 어려웠던 오류 한 가지와 원인 확인 · 수정 · 재검증 과정을 설명하라.

TOKEN_TTL 을 15 초로 바꾸고 20 초 뒤에 글을 썼는데 401 이 아니라 글이 그냥 등록된 것이 가장 헷갈렸습니다. 처음에는 코드가 잘못된 줄 알았는데, AI 에게 물어보니 .env 를 바꾼 뒤 서버를 다시 켜지 않았으면 예전 설정이 그대로라서 그럴 수 있다고 했고, 로그인 응답의 expiresIn 으로 확인하는 방법을 알려 줬습니다. 실제로 저는 .env 를 저장만 하고 백엔드를 재시작하지 않았습니다. 잘못 생긴 글을 지우고 서버를 재시작한 뒤 expiresIn 이 15 로 나오는 것을 확인하고 다시 해 보니, 401 이 나오고 화면도 로그아웃 상태로 바뀌었습니다. 설정 파일은 서버가 켜질 때만 읽는다는 것을 이번에 알게 됐습니다.

현재 남은 문제와 다음에 개선할 점: TTL 이 90 초라서 글을 고치는 도중에 만료되면 다시 로그인해야 했습니다(실제로 테스트 중에 몇 번 겪음). 로그아웃을 해도 이미 복사된 토큰은 만료 전까지 쓸 수 있습니다. 화면에서 403 안내를 직접 띄워 보지는 못했고, 헤더 형식 오류나 iss/aud · exp/sub 가 잘못된 토큰도 직접 만들어 시험해 보지는 못했습니다. 다음에는 만료되기 전에 미리 알려 주는 표시를 넣고, 이런 토큰들도 직접 만들어 시험해 보고 싶습니다.

제출 확인: [x] 실제 코드와 설명 일치 [x] 테스트 직접 수행 [x] AI 활용 및 출처 기록 [x] 비밀 키와 토큰 제거

GitHub 확인: [x] 루트 README 작성 [ ] 보고서 PDF 와 증거 업로드 [ ] README 문서 링크 확인 [x] 교수자 접근 가능(공개 저장소) [ ] 저장소 URL 과 커밋 SHA 제출